

AgentSession and event routing

Prove the value of continuing ownership

Exploratory proposal · November 11, 2026

When a finite run is enough

- Review one pull-request revision.
- Start a task when a Jira issue is created.
- Fan out one advisory into independent remediation workflows.

Webhook → durable router → AgentRun / AgentWorkflowRun

Receiving an event or sharing learnings does not itself require a session.

When continuing ownership may help

Use case	State that spans inputs
PR review	Earlier decisions, human feedback, revisions, pending reviews
CVE remediation	Overlapping advisories, active fixes, exceptions, verification
Jira work	Comments, approvals, delegated outcomes, later instructions

Hypothesis: one continuing owner makes this coordination simpler.

The missing ingress layer

```
Authenticated webhook / poll adapter / human request
  → durable event receipt
  → committed route and destination plan
  → recorded delivery and AgentInput admission
  → controller wake-up and harness worker
```

AgentRouter stays available while session compute sleeps.

It is a component plus route configuration; a public router CRD is not selected.

Commit a receipt before acknowledging

1. Authenticate the source and normalize its identity and intent.
2. Persist the receipt and pinned routing configuration.
3. Return success; process routing and execution asynchronously.

GitHub does not automatically retry failed deliveries.
Jira retries are bounded. Each adapter needs a qualified recovery path.

Event identity and work identity

Identity	Purpose
Provider event key	Recognize retries of one authenticated event
Semantic work key	Coalesce distinct events requesting the same operation
Replay / request ID	Intentionally ask for new work

PR example: repository + PR + head SHA + review purpose.

An issue ID or raw payload hash is not a universal deduplication key.

Commit the route before fan-out

Example: an advisory should reach **20 sessions**.

- Persist the configuration revision, intent, and target UIDs.
- Create 8 inputs, then the router crashes.
- Recover the existing 8 and finish the remaining 12.

Reuse the committed plan even if inventory or routing configuration changed.

Deterministic AgentInput creation

```
inputKey = encode(workKey, targetSessionUID,
                  stableRoute, purpose, replayEpoch, ...)
inputName = "input-" + hash(inputKey)
```

Create by the exact name. On **AlreadyExists**, compare the full identity, canonical intent digest, and target UID.

A match reuses the input. A mismatch is a conflict, never an overwrite.

Close the create-and-record gap

1. Reserve the unique delivery in the durable ledger.
2. Create the input behind an admission gate.
3. Record its resource UID and commit admission.
4. Publish readiness for the session worker.

The input cannot execute or normally disappear before admission is committed.

Late retries, replay, and ordering

- Keep delivery records after AgentInput cleanup.
- Give intentional replay a new explicit identity and lineage.
- State the retention horizon for duplicate suppression.
- Handle out-of-order source revisions with an explicit policy.

One logical input per work identity and destination, within that horizon.

Session worker responsibilities

Claim → restore context → invoke → persist → complete

- Retain earlier decisions and outstanding child references.
- Process a later input while delegated runs continue.
- Correlate child outcomes through durable admission.
- Reuse recorded results; expose uncertain external effects.

Sleep between completed inputs. AgentLearning is an optional integration.

Two gates for the spike

Gate 1 — Reliable event admission

One source adapter, committed routing, duplicate-safe direct-run delivery, partial fan-out recovery, retained ledger, and provider recovery.

Gate 2 — Useful session continuity

Compare finite runs with a session handling a new revision, human follow-up, and out-of-order child results across a restart.

Ingress can succeed independently. Session adoption needs evidence from both.

Decisions still needed

Review now: session-specific use cases and the ingress/delivery contract.

Spike returns: route/work-key policies, admission mechanics, storage/HA, provider coverage, retention, runtime recovery, and operational ownership.

If finite runs with explicit task state are sufficient, use that path.

No dependency on AgentLearning, IDE integration, or VM suspension.

Review material and primary sources

- ADR 0025: `docs/adr/0025-agent-sessions-and-event-routing.md`
- Spike: `docs/spikes/agent-session-event-routing.md`
- GitHub failed webhook deliveries
- Jira Cloud webhook delivery and retries
- Kubernetes object names and identities

Exact provider limits and limitations are in the ingress research note.